LECTURE 3

DATA REPRESENTATIONS

PRESS → OR SPACE.

© Marc Rußwurm
Licensed under **Creative Commons Attribution–NonCommercial 4.0 International (CC BY-NC 4.0)**.
You may share and adapt these slides for teaching, research, and other non-commercial educational purposes with attribution.
Commercial training, paid workshops, consulting seminars, or incorporation into commercial course products require prior permission.

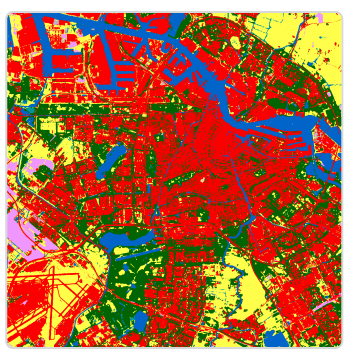
UNIVERSITÄT BONN

Recap: Geospatial Data

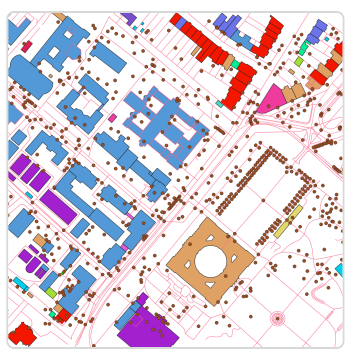
- satellite imagery
- thematic rasters such as land cover
- vector geometries and attributes
- model and reanalysis fields
- observations collected at different times and scales

Each source describes locations on its own spatial and temporal support.

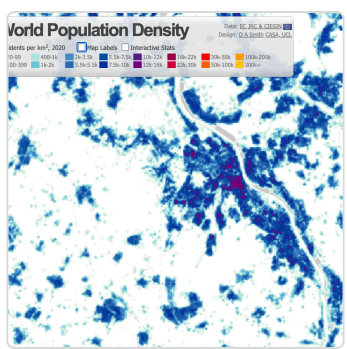
Satellite imagery (Bonn, Sentinel-2)



Thematic raster (land cover, Amsterdam)

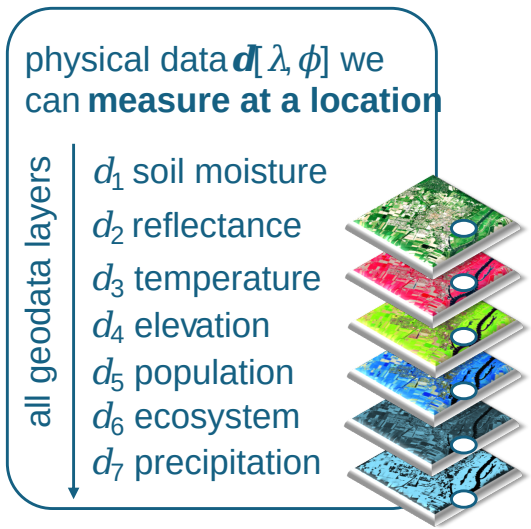


Vector geometries (OpenStreetMap)



Model / reanalysis field (population density)

A Place as a Feature Vector



Sampling all p variables at location i produces a fixed feature vector:

$$\mathbf{x}_i = [x_{i1}, x_{i2}, \ldots, x_{ip}]$$

For example:

$$\mathbf{x}_i = [\text{elevation},\ \text{slope},\ \text{spectral bands},\ \text{temperature},\ \text{building density}]$$

Repeating the extraction at n locations creates a feature matrix:

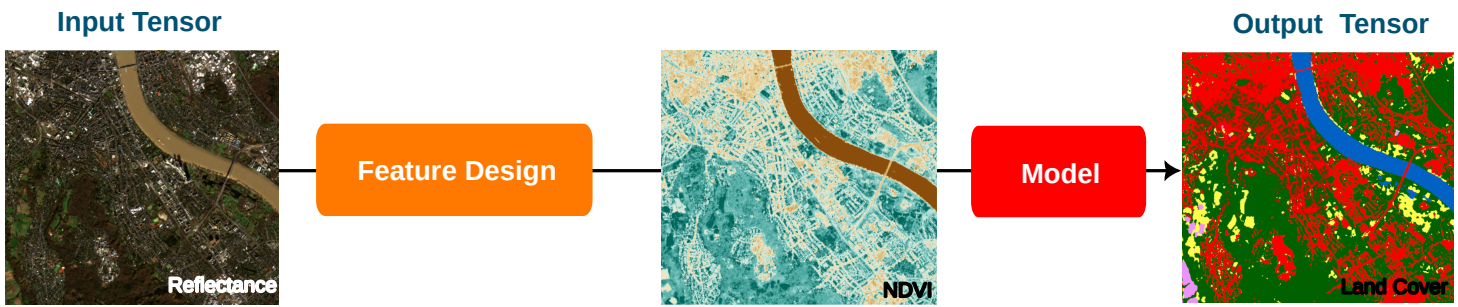
$$\mathbf{X} \in \mathbb{R}^{n \times p}$$

features x_i → machine-learning model → prediction $\hat{y}_i$

Training target y_i: kept separate from the input features.

Good Data Representations for Effective ML

Good, hand-selected features combined with a suitable model determine the quality of the result.

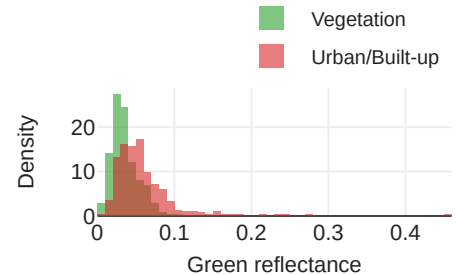


- This lecture · Data Cubes & PCA
- Next lecture · Linear Models & Random Forests

Dataset

Raw bands

Pt	Red	Green	Blue	NIR	Land Cover
Pt1	0.08	0.11	0.09	0.42	**Vegetation**
Pt2	0.21	0.19	0.15	0.05	**Built-up**
⋮	⋮	⋮	⋮	⋮	⋮



Geospatial Data Cubes - Integrating Multi-Modal Data

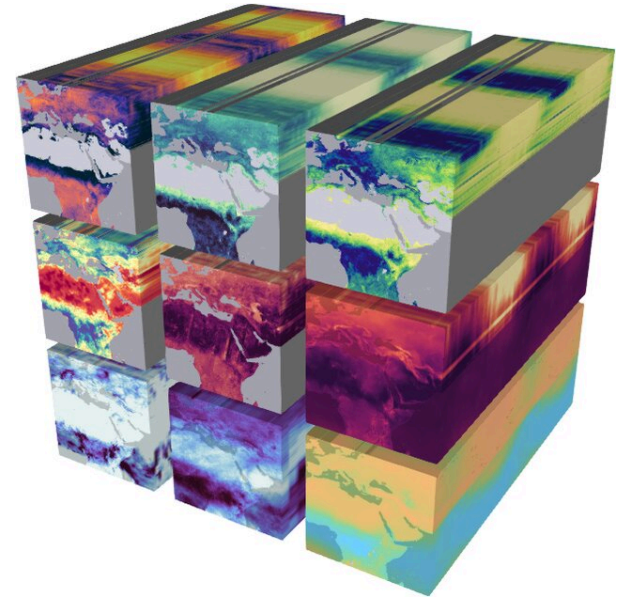
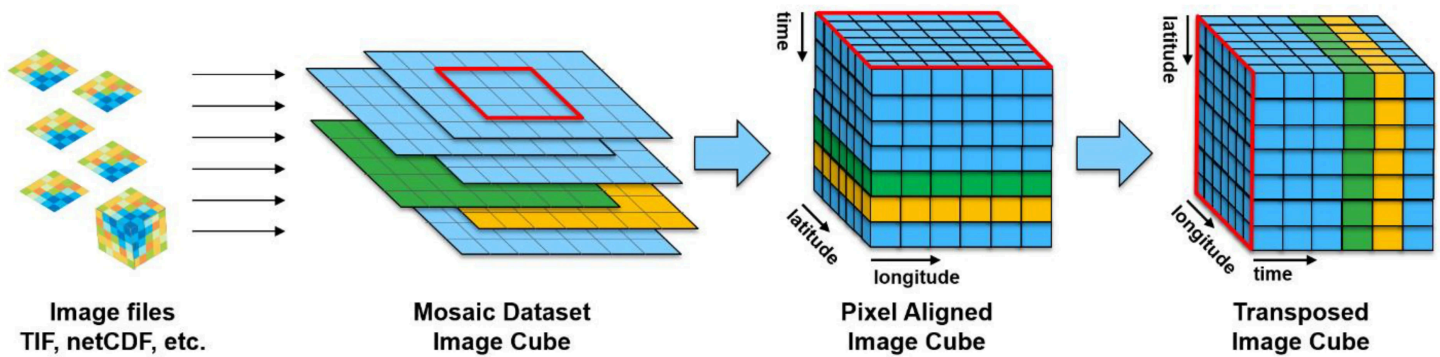


Image by Miguel D. Mahecha

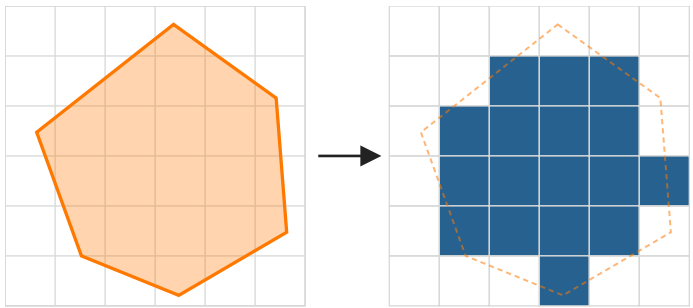
Datacubes align Data Spatially and Temporally



Source: Kopp et al. (2019), ***Achieving the Full Vision of Earth Observation Data Cubes***, *Data* 4(3), 94.

Rasterization and Interpolation

Rasterization

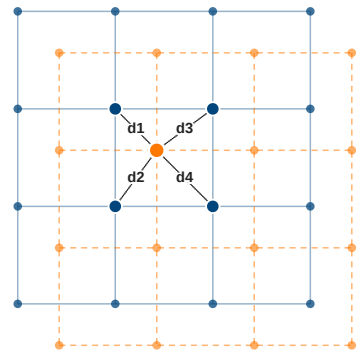


Open-source tools:

`gdal_rasterize` (CLI)

`rasterio.features.rasterize` (Python)

Interpolation



source grid target grid reference point

Open-source tools:

`gdalwarp` / `gdal_translate` (CLI)

`rasterio.warp.reproject` (Python)

Bilinear

2×2 neighborhood

$$\hat{z} = \frac{\sum_{i=1}^{4} z_i / d_i}{\sum_{i=1}^{4} 1 / d_i}$$

Cubic

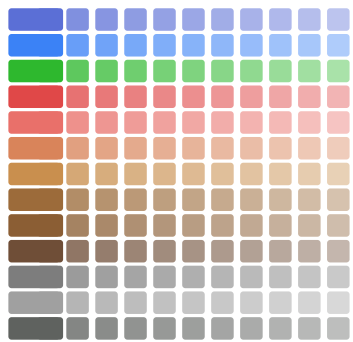
4×4 neighborhood

$$\hat{z} = \frac{\sum_{i=1}^{16} z_i / d_i^3}{\sum_{i=1}^{16} 1 / d_i^3}$$

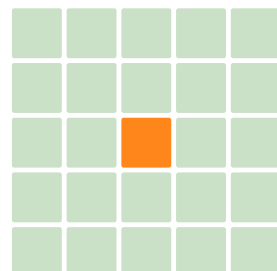
Dimensionality

Sentinel-2 scene around Wageningen (NL), false-colour composite.

Dimensions Add Up Quickly



13 bands × 12 months



5 × 5 neighbourhood

elevation
slope
land cover
population
…

auxiliary layers

dimensions of one feature vector

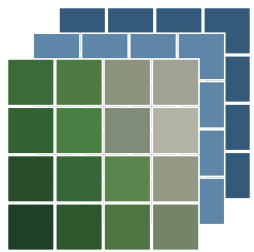
4,225

$D = 13 \times 12 \times 25 + 13 \times 25$

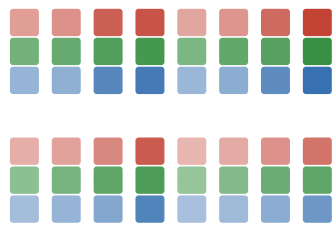
1. one Sentinel-2 pixel — **D = 13**
2. + time: one image per month — **D = 156**
3. + local context: 5 × 5 neighbourhood — **D = 3,900**
4. + auxiliary layers (elevation, slope, land-cover one-hot, …) — **D = 4,225**

Example: An Image as a Dataset of Pixels

3-D tensor:
image (RGB)
`[H × W × C]`



1-D tensors:
list of N = HW pixels
`[C]`



N = 16 vectors with 3 values

2-D tensor:
data(set) matrix
`[N × C]`

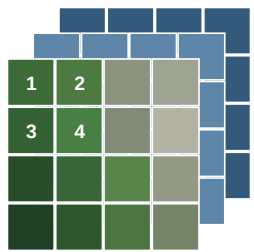
pixel	red	green	blue
1	0.24	0.42	0.22
2	0.30	0.48	0.25
3	0.55	0.58	0.50
4	0.62	0.64	0.58
		⋮	

3 data dimensions
16 data points span a **3-D vector space**

We have **N = HW** pixel data points in 3 dimensions → no information about the pixel neighbourhood.

Example: An Image as One Data Point

3-D tensor:
image (RGB)
`[H × W × C]`



1-D tensor:
stacked pixels
`[HWC]`

“flatten” / “vectorize”

2-D tensor:
data(set) matrix
`[1 × HWC]`

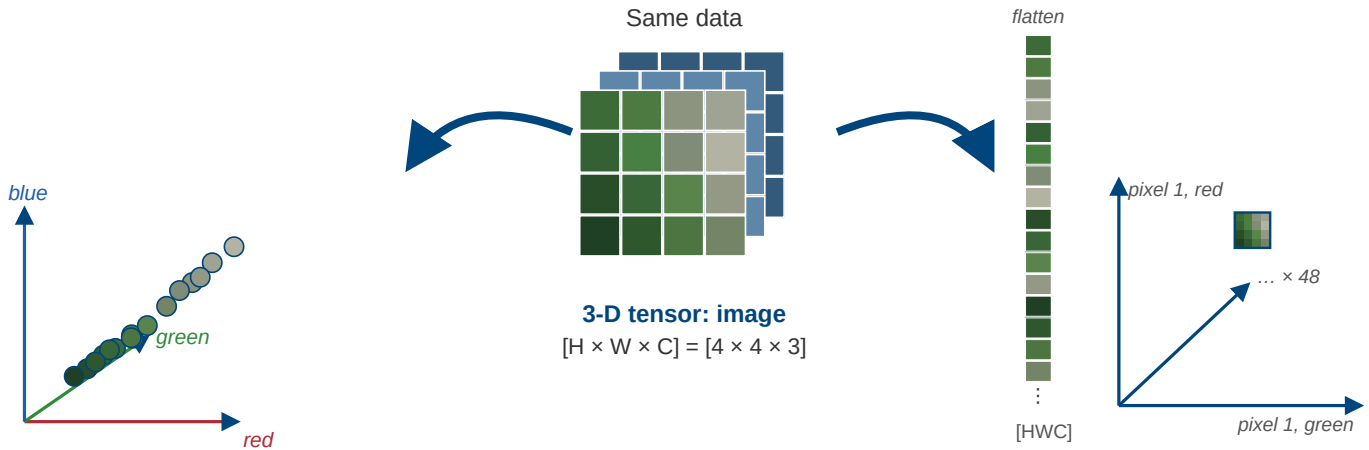
	pixel 1			pixel 2		
image	**red**	**green**	**blue**	**red**	**green**	…
1	0.24	0.42	0.22	0.30	0.48	…

48 = 4 · 4 · 3 data dimensions

one data point spans a **48-D vector space**

We have **one** image data point in 48 dimensions → fewer data points, but the neighbourhood is preserved.

You Design the Vector Space

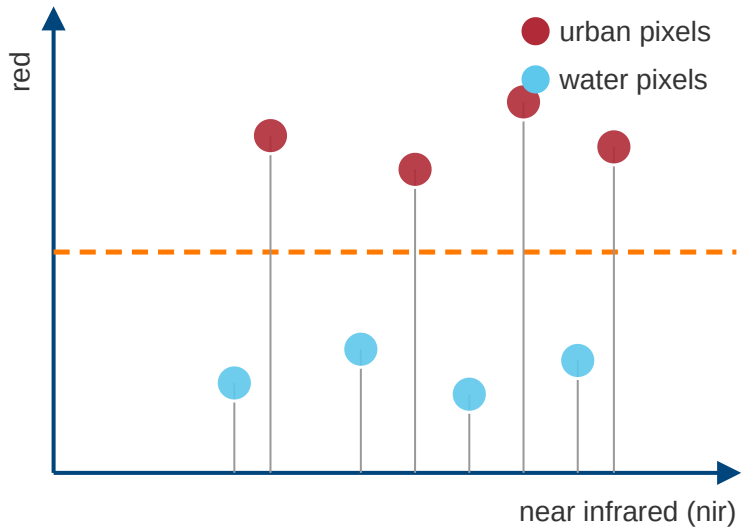


16 data points in 3 dimensions span a **3-D vector space**

1 data point in 48 dimensions span a **48-D vector space**

The same data can easily be represented in few or in many dimensions.

Blessing of Dimensionality: More Information



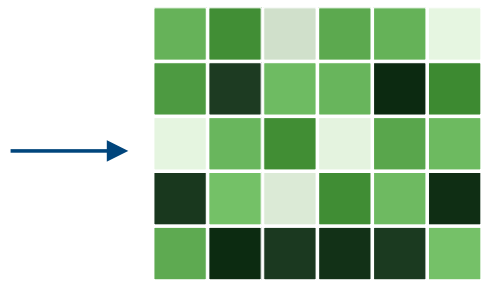
More dimensions can make data separable – good for classification.

Urban and water pixels overlap in the near-infrared band alone.

Data is not separable in nir alone – adding red makes it separable.

Blessing of Dimensionality: Complex Patterns

dataset of 30 pixels: **3-D** RGB vectors



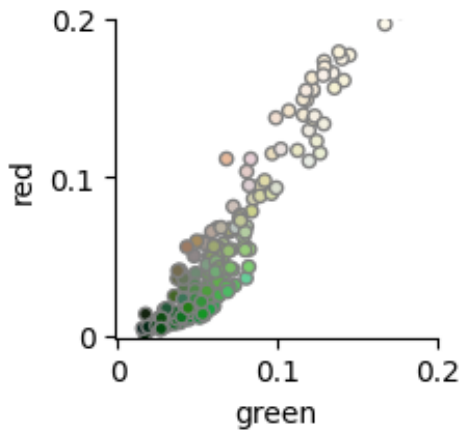
one image: **90-D** vector (30 pixels × 3 channels)

Some **complex patterns** and relationships are missed in a low-dimensional colour space.

Pixel **neighbourhoods** reveal spatial patterns that only emerge in the **high-dimensional** representation.

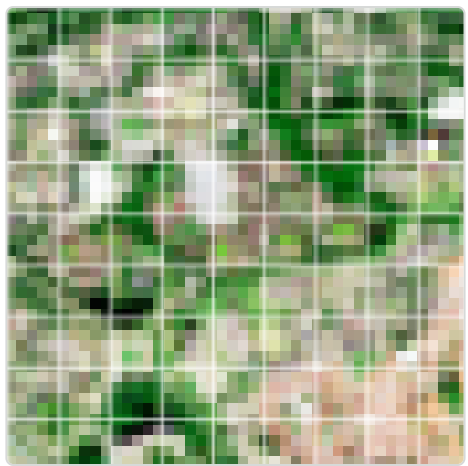
Curse of Dimensionality: Redundancy

Remote sensing bands are highly correlated



Pixels are often bright or dark overall: a very green pixel is usually also very red.

Image pixels are spatially correlated

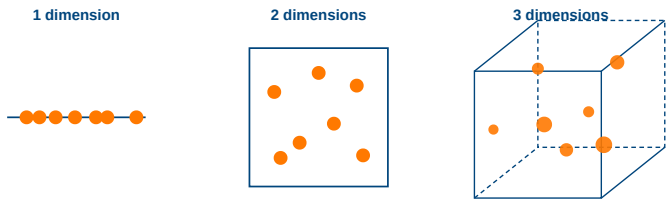


Neighbouring pixels most likely have a very similar colour.

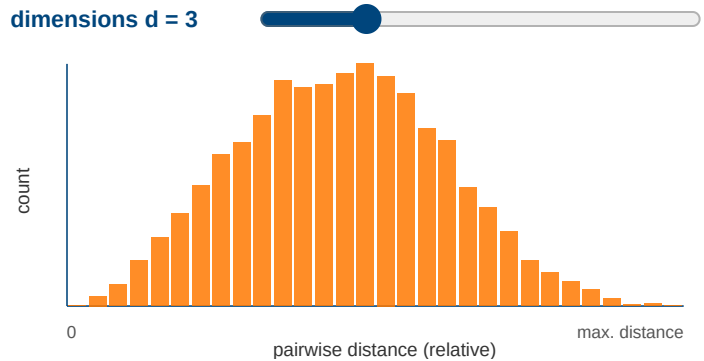
Correlated dimensions add redundancy, not information.

Curse of Dimensionality: Empty Spaces

A higher-dimensional space becomes **emptier** – we need **exponentially more data** to fill it evenly.



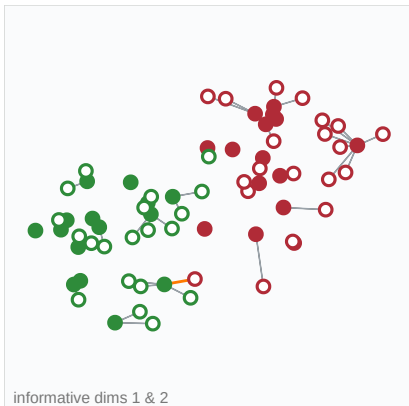
In high dimensions, all samples are **similarly far apart**: distances lose their meaning, and classifiers that rely on "closeness" (e.g. nearest neighbours) become less effective.



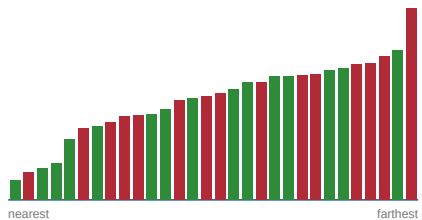
relative contrast $(d_{max} - d_{min}) / d_{min}$ = **10.80** (150 uniform random points)

Curse of Dimensionality: Nearest Neighbours

dimensions d = 2 (0 noise)



distances from one test point to all training points (sorted)



nearest / farthest distance: **0.10**

1-nearest-neighbour test accuracy: **95 %**

/ training

test

wrong neighbour

Example: **nearest-neighbour classification**: a test point gets the class of its closest training point.

In high dimensions, almost all points are **far away — and equally far**.

This weakens the meaning of **“closeness”**: uninformative dimensions drown the informative ones.

> **More dimensions are not automatically more information.**

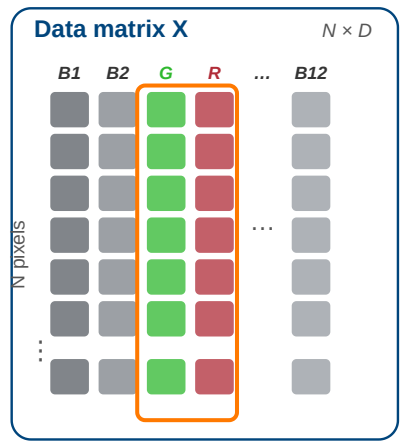
Blessing and Curse of Dimensionality

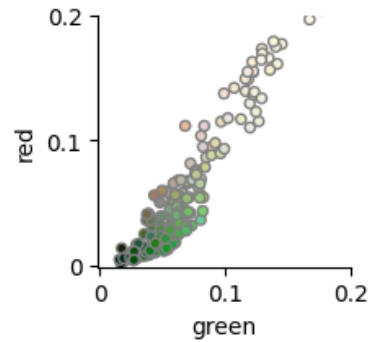
Blessing	Curse
Extra dimensions can provide new, independent information	Correlated (redundant) dimensions add no information
More dimensions help capture complex patterns	More data is needed to fill the emptier high-dimensional space
	Distances lose meaning; classifiers become less effective

Keep as many dimensions as necessary, but as few as possible.

Bands as Variables: The Data Matrix

13 Sentinel-2 bands

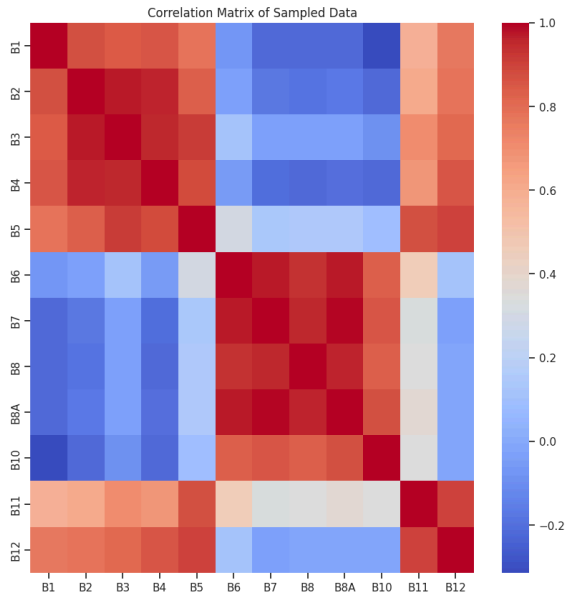




green vs. red reflectance per pixel

Reflectance data has high covariance and correlation between bands.

Sentinel-2 Bands Are Highly Correlated



Correlation matrix of 12 Sentinel-2 bands (red = +1, blue = −1)

- **visible** bands (B1–B5) are highly correlated with each other
- visible bands are **not** correlated with **near-infrared** (B6–B9)
- visible bands are again correlated with **SWIR** (B11, B12)

Many of our dimensions carry (almost) the same information.

Reducing Dimensions: Two Approaches

Keep as many dimensions as necessary, but as few as possible.

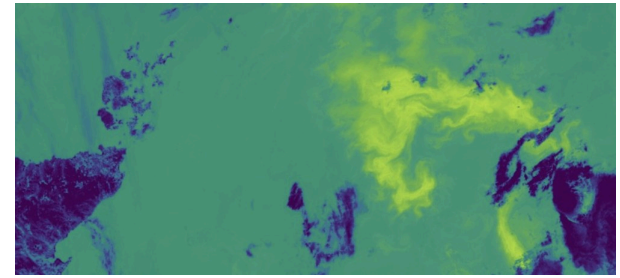
1 · Feature selection (domain knowledge)

- design features that summarise correlated bands: spectral indices (NDVI, SAVI, EVI, …), radar ratios
- automatically select the most important features

2 · Dimensionality reduction (domain-agnostic)

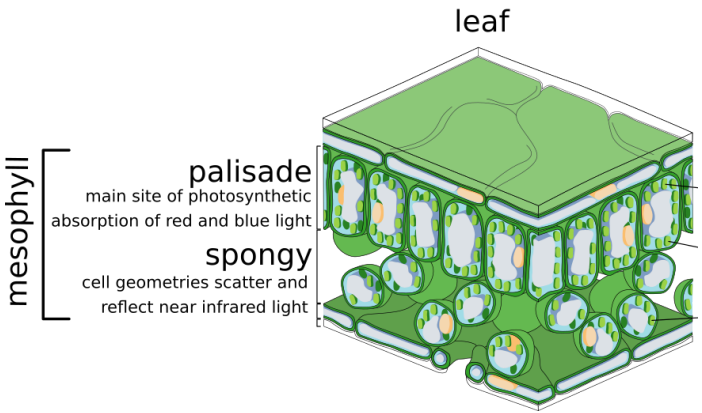
- project the data into fewer, less correlated, more informative dimensions
- algorithms: **Principal Component Analysis (PCA)**, Minimum Noise Fraction (MNF), …

Domain Knowledge: Feature Design

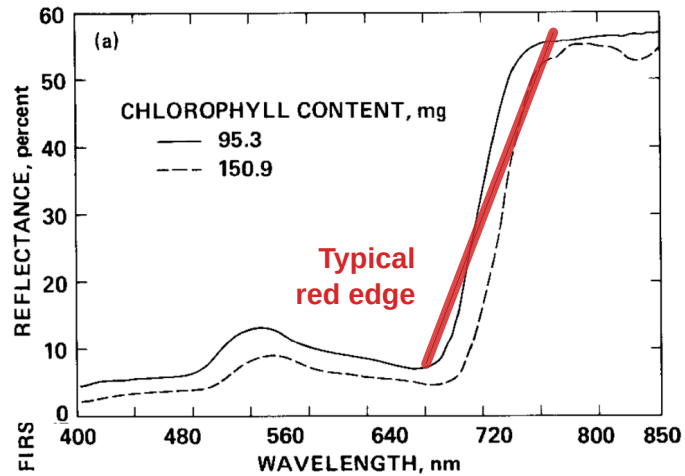


Chlorophyll-a index from Sentinel-3 OLCI, algal bloom off Scotland (2013-06-14).

Recap: Vegetation Spectra



1. **absorption** of blue (≈ 450 nm) and red (≈ 670 nm) light by chlorophyll a/b
2. strong **reflectance** of near-infrared light (> 750 nm) due to scattering in the leaf structure

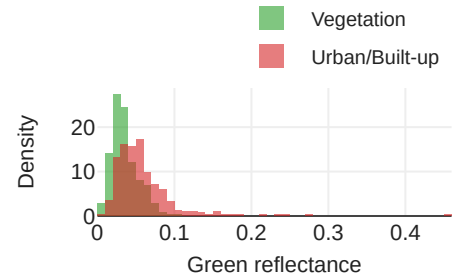


Curran, P. J., Dungan, J. L., & Gholz, H. L. (1990). Exploring the relationship between reflectance red edge and chlorophyll content in slash pine. *Tree Physiology*, 7(1-4), 33–48.

Decorrelation by Domain Knowledge: NDVI

Raw bands

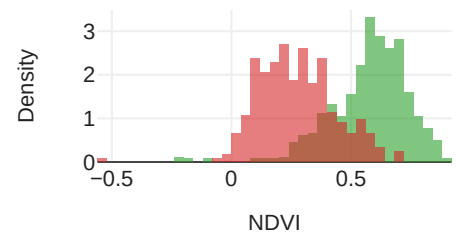
Pt	Red	Green	Blue	NIR	Land Cover
Pt1	0.08	0.11	0.09	0.42	**Vegetation**
Pt2	0.21	0.19	0.15	0.05	**Built-up**
⋮	⋮	⋮	⋮	⋮	⋮



Engineered feature

$$\text{NDVI} = \frac{\text{NIR} - \text{Red}}{\text{NIR} + \text{Red}}$$

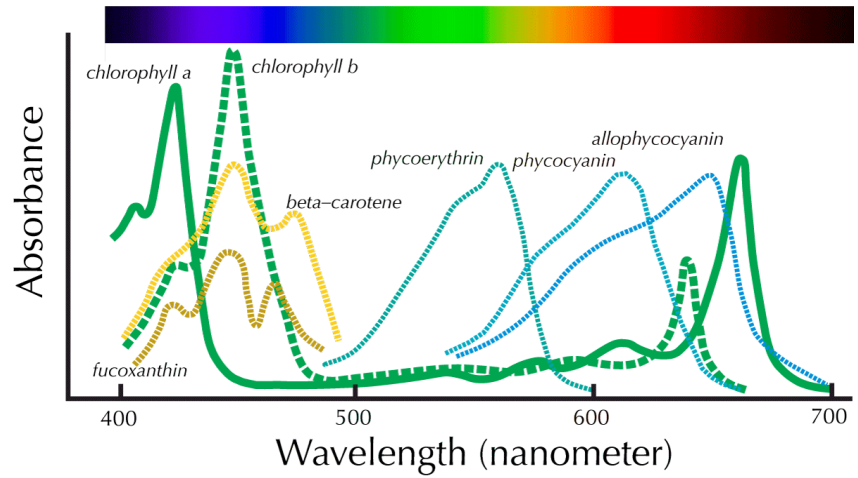
Pt	NDVI	Land Cover
Pt1	0.68	**Vegetation**
Pt2	-0.62	**Built-up**
⋮	⋮	⋮



Photosynthetically Active Pigments

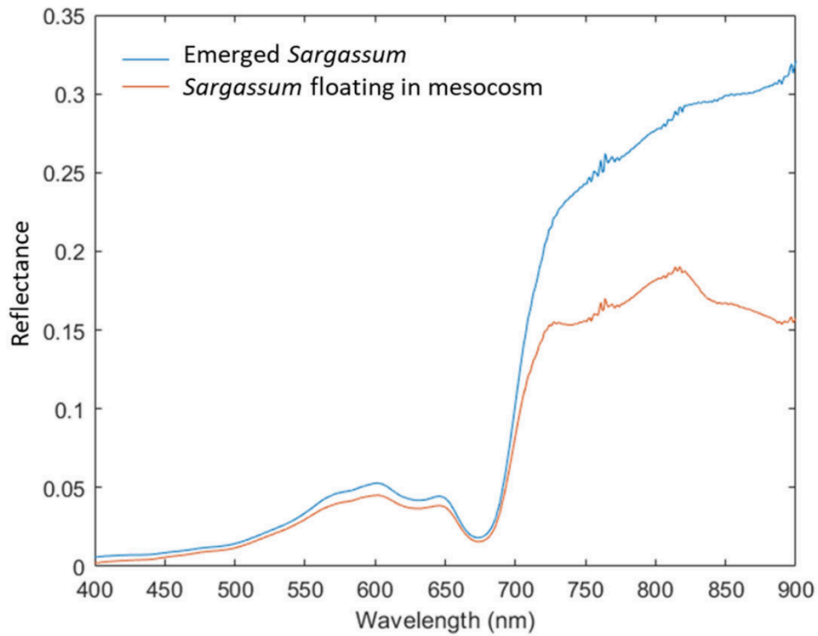
- **chlorophylls** and other pigments capture the energy of light
- different organisms use **different pigments** for photosynthesis
- terrestrial plants use both **CHL-a and CHL-b**
- algae and cyanobacteria do not: e.g. **phycocyanin**, phycoerythrin, fucoxanthin

Each pigment leaves its own fingerprint in the spectrum.



Source: York University, Photosynthesis lab manual

The Red Edge Also Appears in Algae

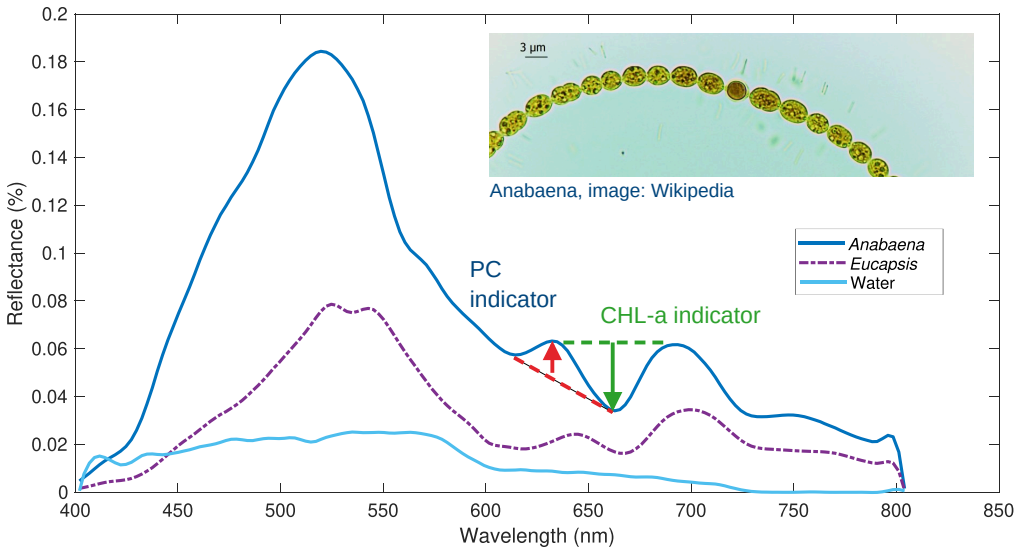


Descloitres, J., Minghelli, A., Steinmetz, F., Chevalier, C., Chami, M., & Berline, L. (2021). Revisited estimation of moderate resolution Sargassum fractional coverage using decametric satellite data (S2-MSI). *Remote Sensing*, 13(24), 5106.

Sargassum (macroalgae)

The same **red edge** as land vegetation: the vegetation signal is not limited to land.

Indicators for CHL-a and Phycocyanin (PC)



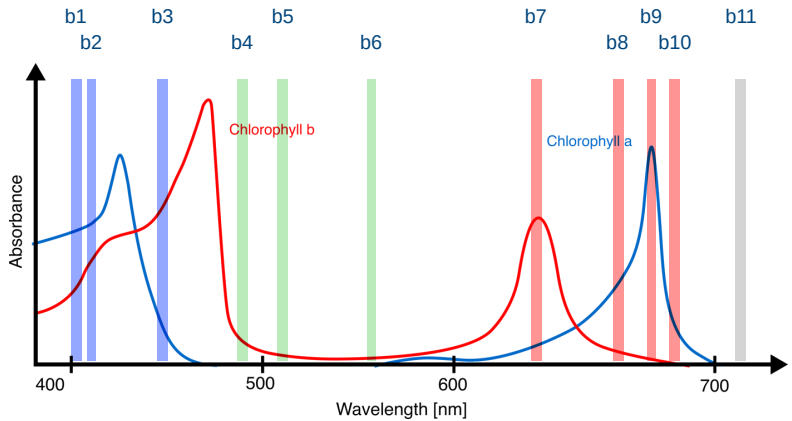
Modified from: Legleiter, C. J., et al. (2022). Spectral mixture analysis for surveillance of harmful algal blooms (SMASH). *Remote Sensing of Environment*, 279, 113089.

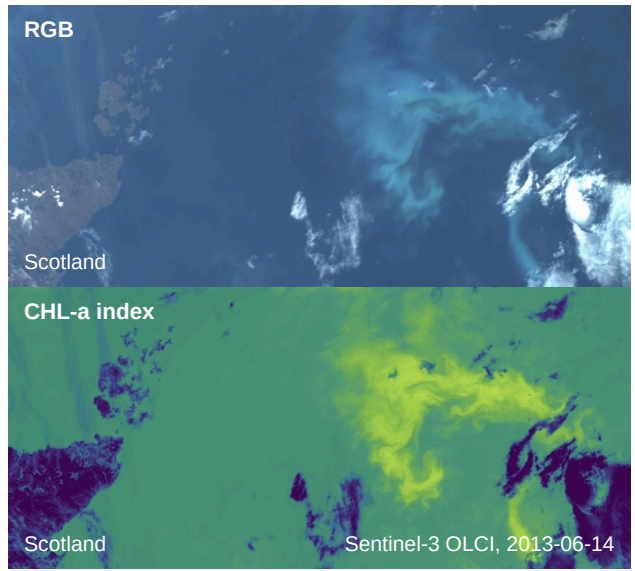
Eucapsis, image: Jason Oyadomari, Michigan Technological University, USA

The **shape** of the spectrum around 620–700 nm distinguishes pigments.

Exercise: Design a CHL-a Index

Sentinel-3 OLCI bands overlaid with chlorophyll absorbance



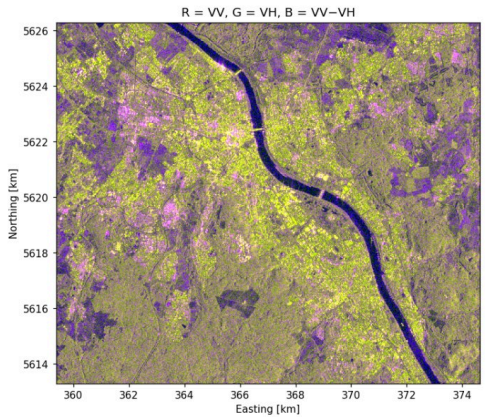


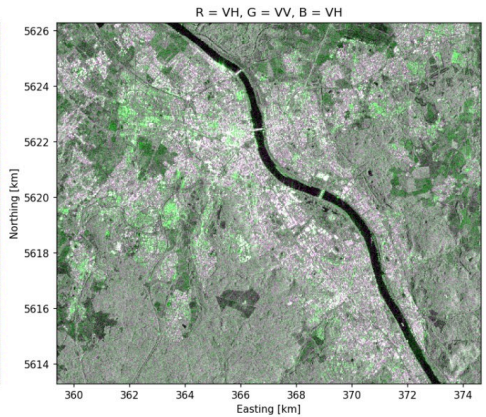
Pick two OLCI bands and combine them like NDVI, as a **normalized difference**:

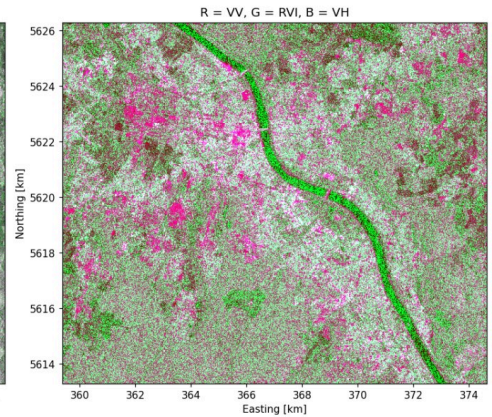
$$\text{index} = \frac{b_i - b_j}{b_i + b_j}$$

Exercise script (Google Earth Engine) · Solution

Radar Features: Sentinel-1 Polarizations







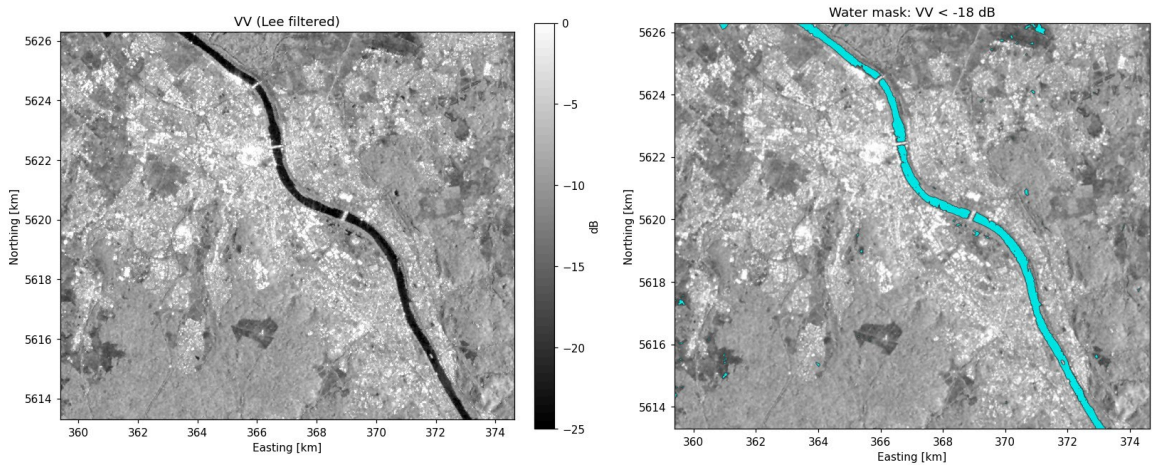
VV (co-polarized): surface and double-bounce scattering → bright in **built-up** areas

VH (cross-polarized): volume scattering → sensitive to **vegetation** structure

designed features: $\mathrm{VV} - \mathrm{VH}$ (a ratio in dB), radar vegetation index $\mathrm{RVI} = \frac{4\,\mathrm{VH}}{\mathrm{VV}+\mathrm{VH}}$

Sentinel-1 subset around Bonn · figures: Lab 2 (Geodata Scavenger Hunt), Group 2

Radar: From Speckle Filter to Water Mask



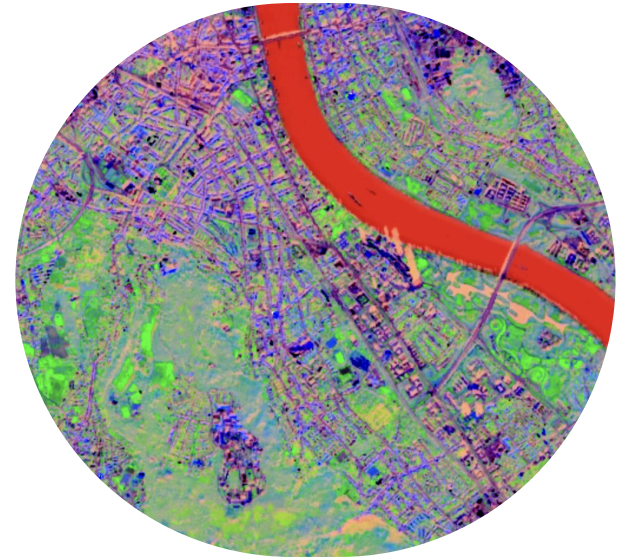
1 · Lee filter: removes the grainy **speckle** of SAR images by local averaging, adapted to the local variance → smooths homogeneous areas, keeps edges

2 · Threshold: calm water reflects the signal away from the sensor → very low backscatter: $\mathrm{VV} < -18\,\mathrm{dB} \Rightarrow$ water

Result: one designed feature plus one rule gives a **water mask** – no training data needed

Figures: Lab 2 (Geodata Scavenger Hunt), Group 2

PCA: Decorrelating Data Algorithmically



Sentinel-2 scene of Bonn, first three principal components as RGB.

(Image) Statistics 1: A Single Variable

Statistical moments of one band *d*

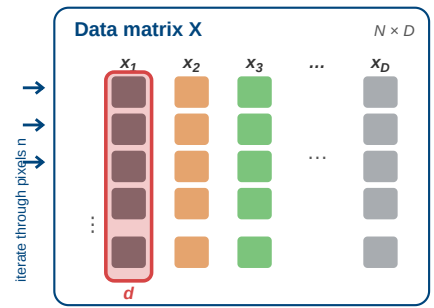
1st moment: mean

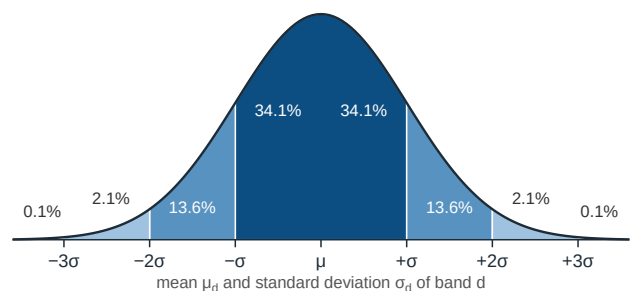
$$\mu_d = \frac{1}{N}\sum_{n=1}^{N} x_{n,d}$$

2nd moment: variance

$$\sigma_d^2 = \frac{1}{N-1}\sum_{n=1}^{N}(x_{n,d} - \mu_d)^2$$

3rd moment: skewness · **4th moment: kurtosis**





(Image) Statistics 2: Two Variables

Statistical moments of two bands d_1, d_2

1st moment: mean vector

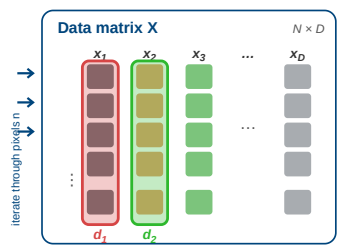
$$\boldsymbol{\mu} = \begin{pmatrix} \mu_{d_1} \\ \mu_{d_2} \end{pmatrix}, \quad \mu_d = \frac{1}{N} \sum_{n=1}^{N} x_{n,d}$$

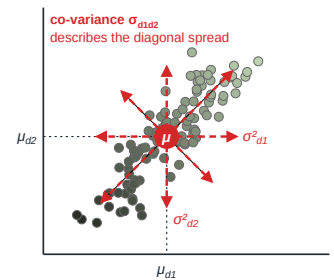
2nd moment: covariance matrix

$$\mathbf{C} = \begin{pmatrix} \sigma_{d_1}^2 & \sigma_{d_1 d_2} \\ \sigma_{d_2 d_1} & \sigma_{d_2}^2 \end{pmatrix}$$

with the **covariance**

$$\sigma_{d_1 d_2} = \sigma_{d_2 d_1} = \frac{1}{N-1} \sum_{n=1}^{N} (x_{n,d_1} - \mu_{d_1})(x_{n,d_2} - \mu_{d_2})$$





(Image) Statistics 3: Many Variables

Statistical moments of all *D* bands

1st moment: mean vector

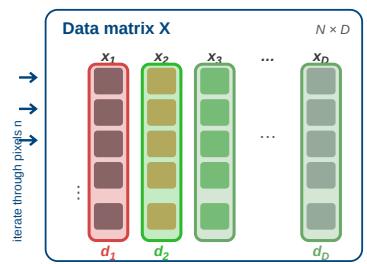
$$\boldsymbol{\mu} = (\mu_1, \mu_2, \ldots, \mu_D)$$

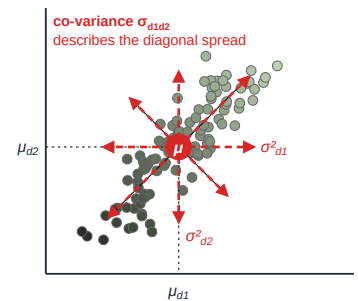
It is advised to **center** the data matrix:

$$\tilde{\mathbf{X}} = \mathbf{X} - \boldsymbol{\mu}$$

2nd moment: covariance matrix $(D \times D)$

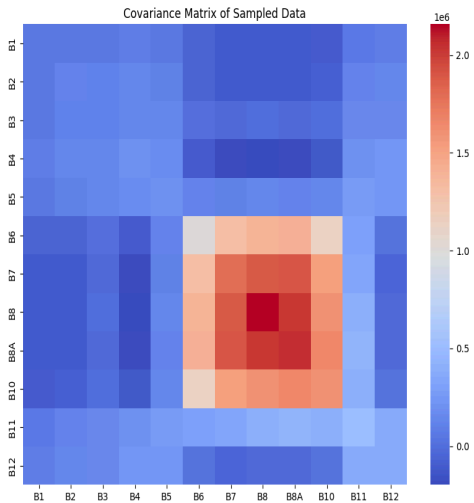
$$\mathbf{C} = \begin{pmatrix} \sigma_1^2 & \cdots & \sigma_{1D} \\ \vdots & \ddots & \vdots \\ \sigma_{D1} & \cdots & \sigma_D^2 \end{pmatrix} = \frac{1}{N-1}\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}$$



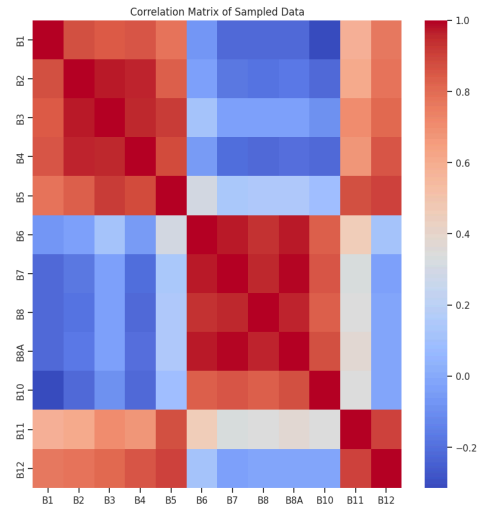


every pair of bands has such a covariance

Covariance vs. Correlation Matrix



Covariances are dominated by bands with large variance (NIR).

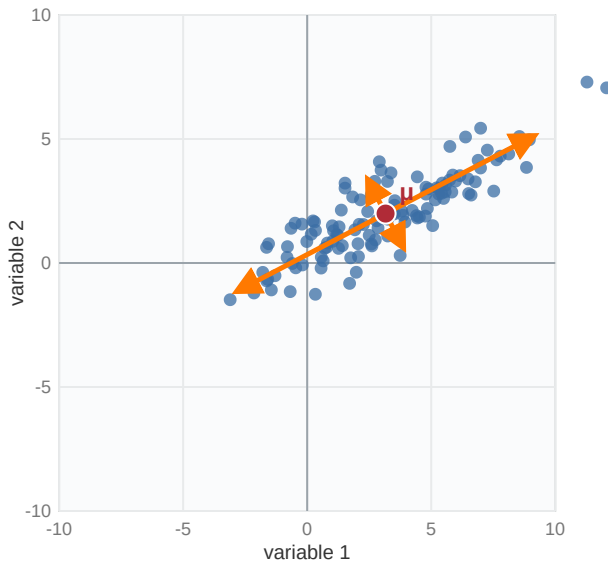


Correlation = normalised covariance

$$\rho_{d_1 d_2} = \frac{\sigma_{d_1 d_2}}{\sigma_{d_1}\,\sigma_{d_2}} \in [-1, 1]$$

Correlation is the **normalised** covariance: it removes the effect of the band variances and makes bands comparable.

Which Covariance Do We Want?



correlation ρ 0.85

spread σ_1 3.0

spread σ_2 1.8

1 · translate (center) | 2 · rotate to PCs | reset

☑ show principal axes

Sample covariance matrix C

$$\begin{pmatrix} 9.10 & 4.45 \\ 4.45 & 2.93 \end{pmatrix}$$

mean μ = (3.16, 1.99)

eigenvalues $\lambda_1 = 11.43$, $\lambda_2 = 0.60$ → PC 1 explains **95 %**

For a high **information density**, we want **small covariances**. We can **translate** (center) and **rotate** the data so that the covariances vanish: this is **Principal Component Analysis**.

PCA: A Linear Orthogonal Transformation

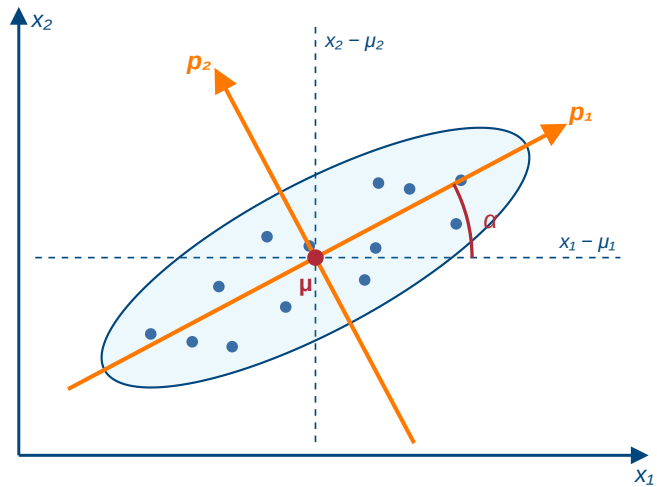
1. **translation**: $\tilde{\mathbf{X}} = \mathbf{X} - \boldsymbol{\mu}$
2. **rotation** by angle α

D original **correlated** variables $x_1, \ldots, x_D$

↓ **PCA**

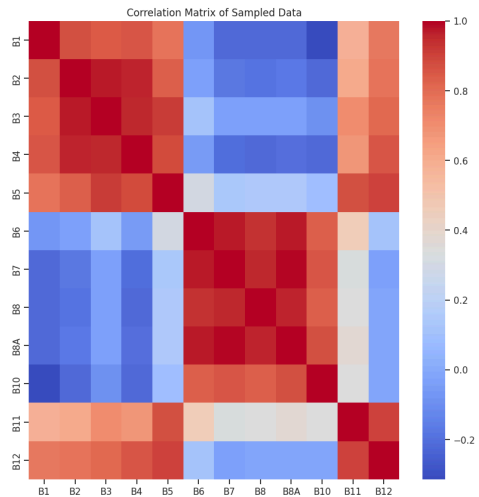
D new **uncorrelated** variables $p_1, \ldots, p_D$

Translation and rotation do not lose information: with all *D* components, the data can be fully reconstructed.



PCs Are Uncorrelated “Synthetic Bands”

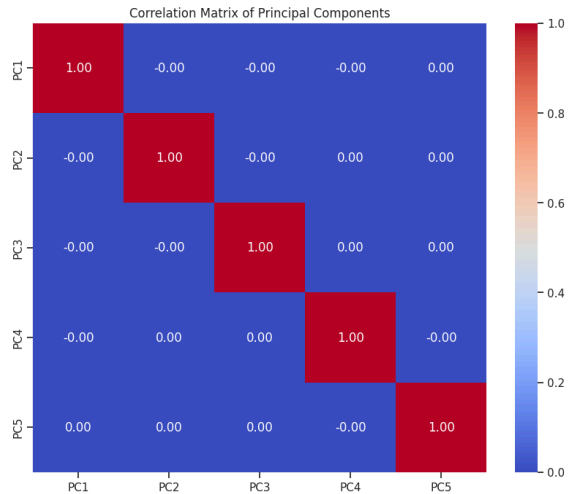
12 correlated Sentinel-2 bands



PCT

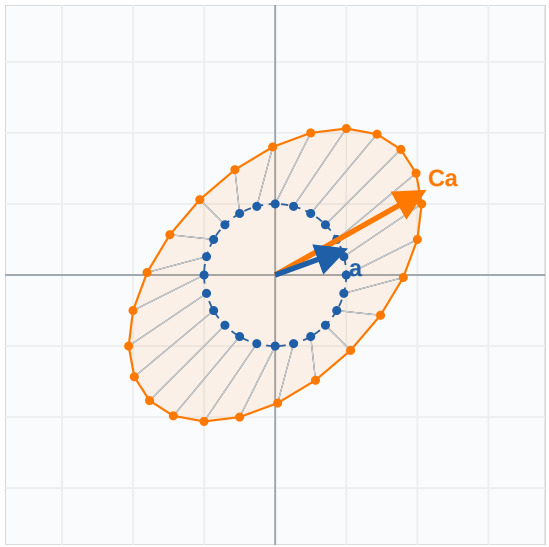
principal component transformation

(fewer) uncorrelated principal components



Same goal as a spectral index – remove redundancy – but found **from the data alone**, without knowing what the bands mean.

Covariance as a Linear Transformation



Covariance matrix C

(2, 0.5; 0.5, 2) (2, 0; 0, 1) (3, 1.2; 1.2, 1)

direction of **a** 20°

a = (0.94, 0.34)
Ca = (2.05, 1.15)

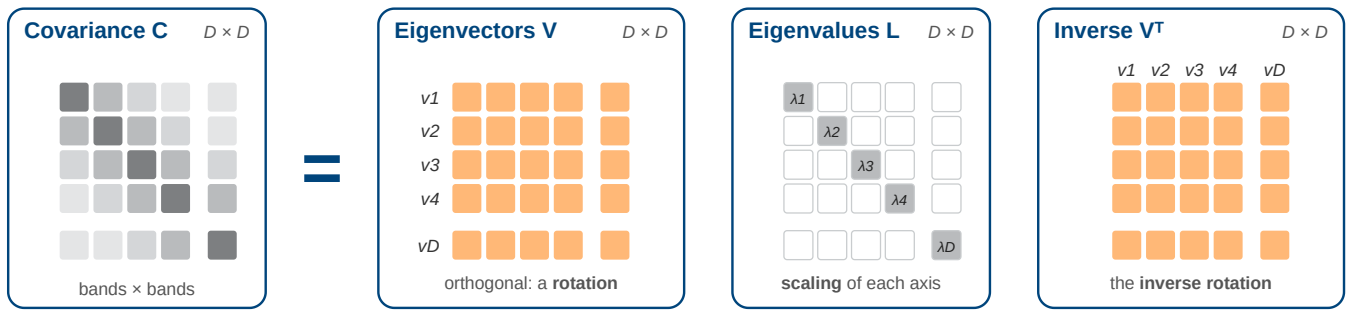
Ca points in a different direction than a.

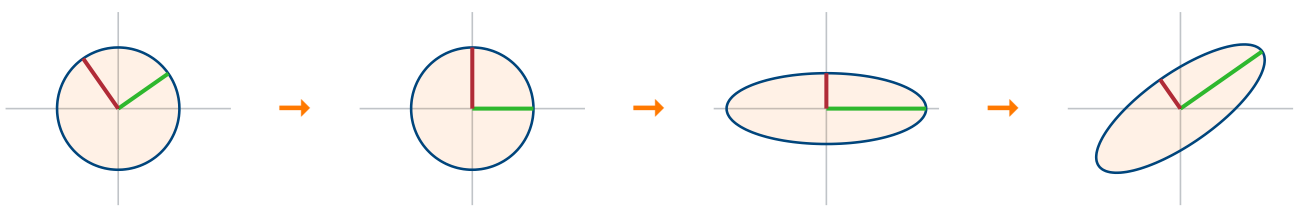
☐ show eigenvectors

Applied to a unit circle, $\mathbf{C}$ describes the **spread of the data**: $\mathbf{x} = \mathbf{Ca}$.

Eigenvectors $\mathbf{v}$ and **eigenvalues λ** of $\mathbf{C}$:
$\mathbf{Cv} = \lambda\mathbf{v}$

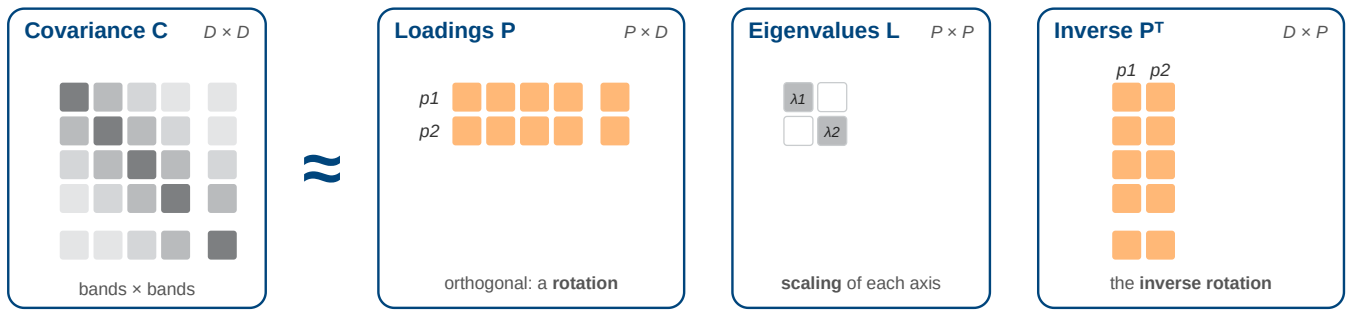
Eigendecomposition: Rotation and Scaling

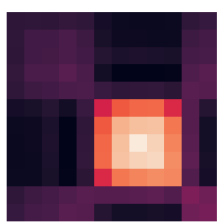




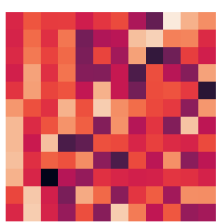
$\mathbf{C} = \mathbf{V}\mathbf{L}\mathbf{V}^\top$: the rotation $\mathbf{V}^\top\tilde{\mathbf{x}}$ **decorrelates** the data, and the **eigenvalues are the variances** in the rotated space.

Few Large Eigenvalues: Principal Components

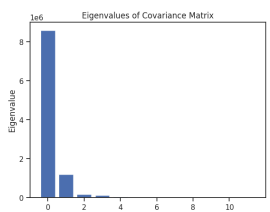




covariance of 12 bands



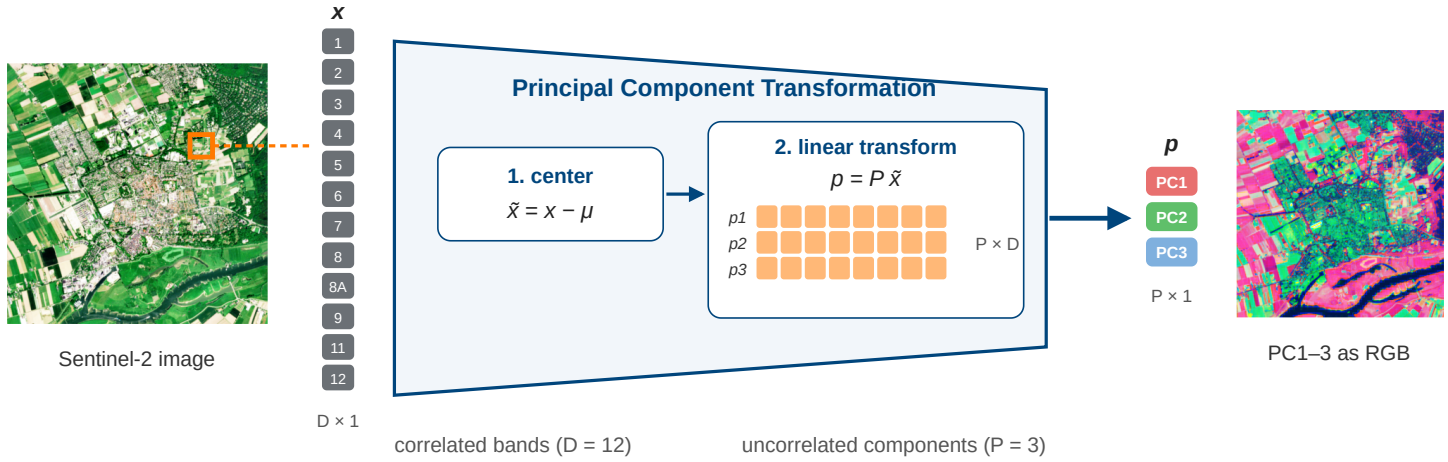
eigenvectors



eigenvalues: only few are large

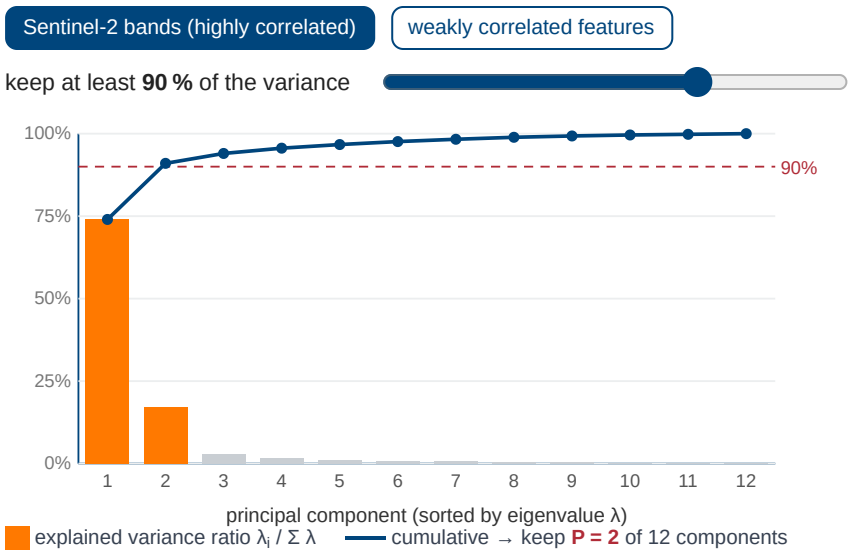
If bands are strongly correlated, **most eigenvalues are close to 0**: keep only the $P \ll D$ eigenvectors with the largest eigenvalues, the **principal components**.

Principal Component Transformation (PCT)



Once the eigenvectors are known: **one matrix multiplication** per pixel.

How Many Principal Components to Choose?



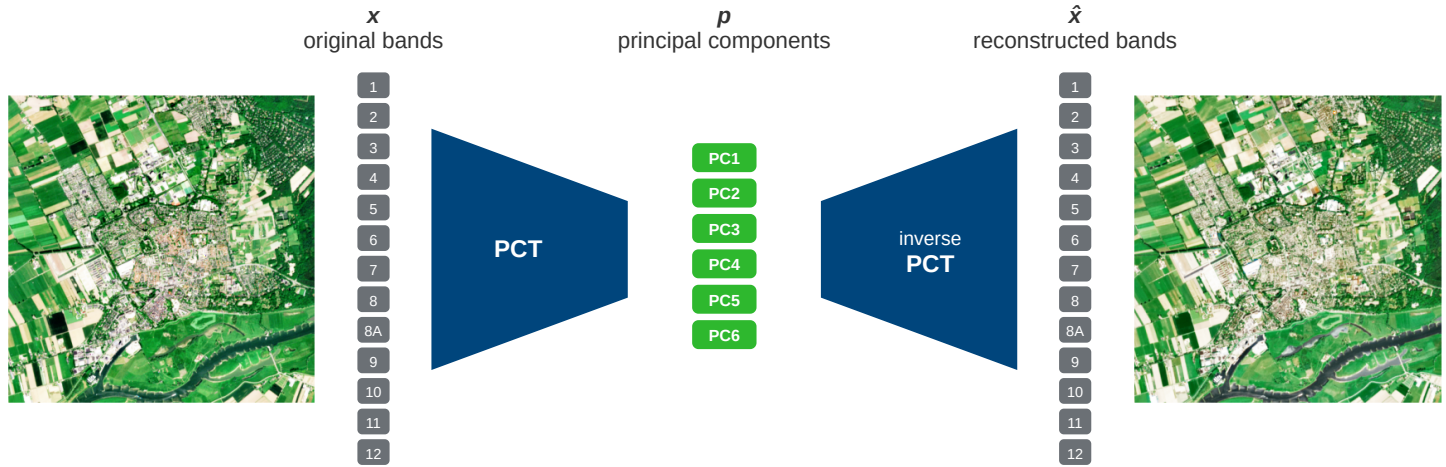
It depends on the **correlation** in the data: the stronger the correlation, the fewer components are needed.

1. sort the eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots$
2. compute the **explained variance ratio** $\lambda_i / \sum_j \lambda_j$
3. choose P so that the cumulative ratio exceeds e.g. **90 %, 95 % or 99 %**

Exercise: eigenvalues (0.4, 0.3, 0.2, 0.1, 0.0). How many PCs for 90 %?

Principal Components “Densify” Information

by removing the correlation between bands

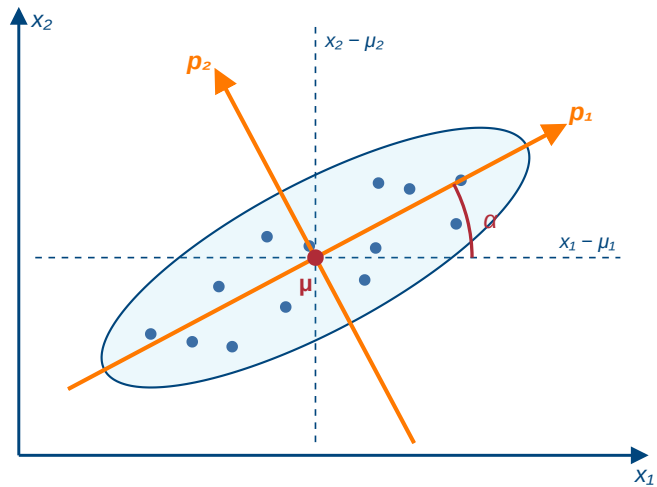


6 principal components reconstruct the 13-band image with **99 % “explained variance”**

Summary: Calculating Principal Components

The **principal components** $\mathbf{p}_1, \mathbf{p}_2, \ldots$ are the **eigenvectors** with the largest **eigenvalues** of the **covariance matrix** $\mathbf{C}$.

1. center the data matrix: $\tilde{\mathbf{X}} = \mathbf{X} - \boldsymbol{\mu}$ (N pixels × D bands)
2. calculate the $D \times D$ covariance matrix $\mathbf{C}$
3. calculate eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_D$ and eigenvalues $\lambda_1, \ldots, \lambda_D$
4. sort by eigenvalue and keep the $P \ll D$ eigenvectors with the largest λ
5. project $\tilde{\mathbf{X}}$ onto the principal components to obtain the reduced dataset



What Do We Get? Visualization and Analysis

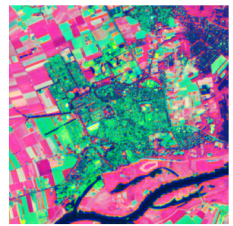
13 Sentinel-2 bands

PCT

PC1

PC2

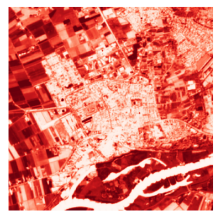
PC3



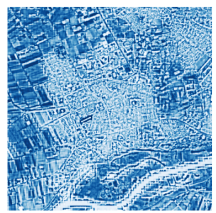
Purpose 1: Visualization
the 3 most informative principal components shown as red, green and blue

Purpose 2: Analysis

PC1 – “vegetation”



PC2 – “built-up & veg.”



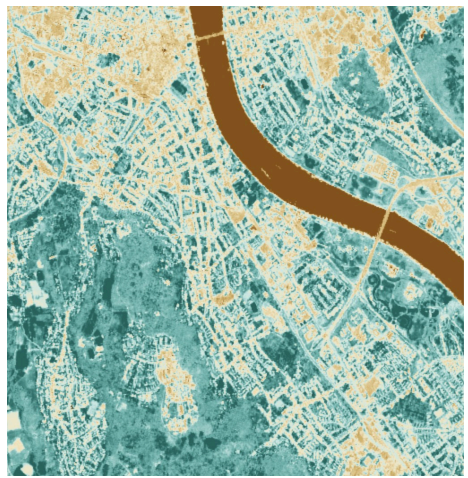
PC3 – “built-up”

Purpose 1: Visualization

True colour: 3 of 12 bands

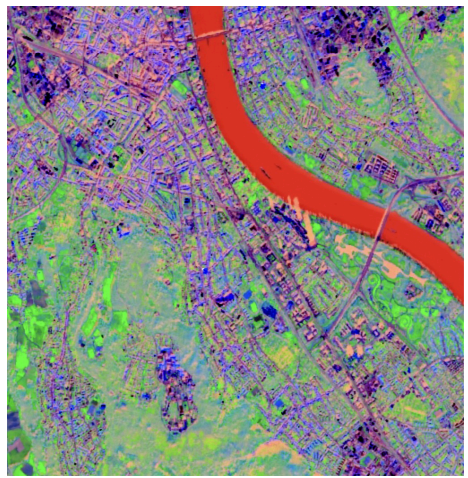
Leaf-off forest and dense city both look dark.

NDVI: one *designed* feature



Vegetation (teal) vs. built-up (beige) vs. water (brown) – one dimension, interpretable.

PC 1–3 as RGB: *learned* features



Water, grassland, forest and built-up separate at once – no expert knowledge needed, but no physical unit.

Purpose 2: Analysis

PC1 – “grass/crop & healthy vegetation”

PC2 – “built-up / bare soil”

Just **2 principal components** already yield a basic land-cover map.

Curse of dimensionality: with few labelled data points, classification and regression work much better on fewer dimensions.

PCA in Practice

```python
import numpy as np
from sklearn.decomposition import PCA

# demo data: 100 samples, 5 features
X = np.random.rand(100, 5)

# initialize the PCA transform object
pca = PCA(n_components=2)

# fit (eigendecomposition) and apply the transformation
pc_features = pca.fit_transform(X)

pc_features.shape              # (100, 2)
pca.explained_variance_ratio_  # per component
```

Fitting PCA on thousands of pixels with a dozen bands takes **milliseconds**.

Once fitted, transforming new data is **a single matrix multiplication**.

> **The tedious part is computing by hand – not for the computer.**

FYI – PCA on whole images in Google Earth Engine: eigen-analysis guide · example script

PCA: Advantages and Limitations

Advantages

- **dimensionality reduction:** fewer features that keep most of the variance
- **noise reduction:** dropping small components filters noise
- **efficiency:** less memory, faster training
- **uncorrelated features:** can improve model performance
- **visualization:** 2–3 components fit into a plot or an RGB image

Limitations

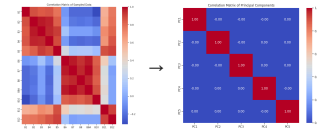
- **loss of interpretability:** components have no unit and mix all bands
- **assumes linearity:** captures only linear correlations
- **sensitive to scaling:** features with large values dominate → normalize (less critical for reflectances in the same unit)
- **sensitive to outliers:** outliers pull the principal axes
- **loss of information:** fine-grained information in small components is lost

PCA is one of the most widely used data analysis tools: remote sensing, image compression, genomics, social sciences, recommender systems.

Summary and Core Concepts

Statistics
covariances & correlations

Correlated bands are **redundant**; PCA translates and rotates the data so that the components are **uncorrelated**.



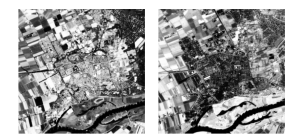
Linear algebra
eigenvalue decomposition

The principal components are the **eigenvectors** of the covariance matrix; the eigenvalues are their variances. Applying PCA is **one matrix multiplication**.

```
p = P(x - μ)
```

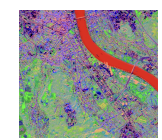
Machine learning
dimensionality reduction

Few principal components keep most of the variance and serve as compact **features**, e.g. for land-cover classification.



Remote sensing
multi-spectral visualization

Three PCs show high-dimensional Sentinel-2 data in **one RGB image**.



Domain knowledge (indices) and PCA both reduce correlated dimensions: **indices are interpretable**, **PCA needs no expert knowledge**.